



Development of an Enterprise Infrastructure Management System (EIMS)

Student: Ratthabhum Peansukmanee (Student ID: 66011198) | Project Advisor: Asst. Prof. Dr. Phongsak Keeratiwintakorn

Host Enterprise: KANEMATSU ELECTRONICS (THAILAND) | Workplace Supervisor: Mr. Chalermpol Phuangchalam

1. Abstract

Enterprise infrastructure auditing across isolated corporate networks is hindered by air-gapped endpoints and fragmented diagnostic records. When failures occur on hosts lacking persistent agents, administrators face manual triage without verifiable traceability. The Enterprise Infrastructure Management System (EIMS) addresses this challenge through an integrated evidence management and AI-assisted investigation platform. EIMS combines a central PostgreSQL asset registry with a portable USB collector powered by embedded CPython 3.14.3, extracting non-destructive WMI telemetry and Windows Event Logs (System and Application channels, default maximum 500 events) into schema-validated JSON reports. Upon ingestion, sequential SHA-256 deduplication and bounded semantic retrieval triage prioritized events (cap <= 10). Empirical validation confirmed end-to-end data flow across 49 persisted field events, 10 AnalysisHistory records, HTTP 200 API responses, and browser runtime verification (A9 PASS).

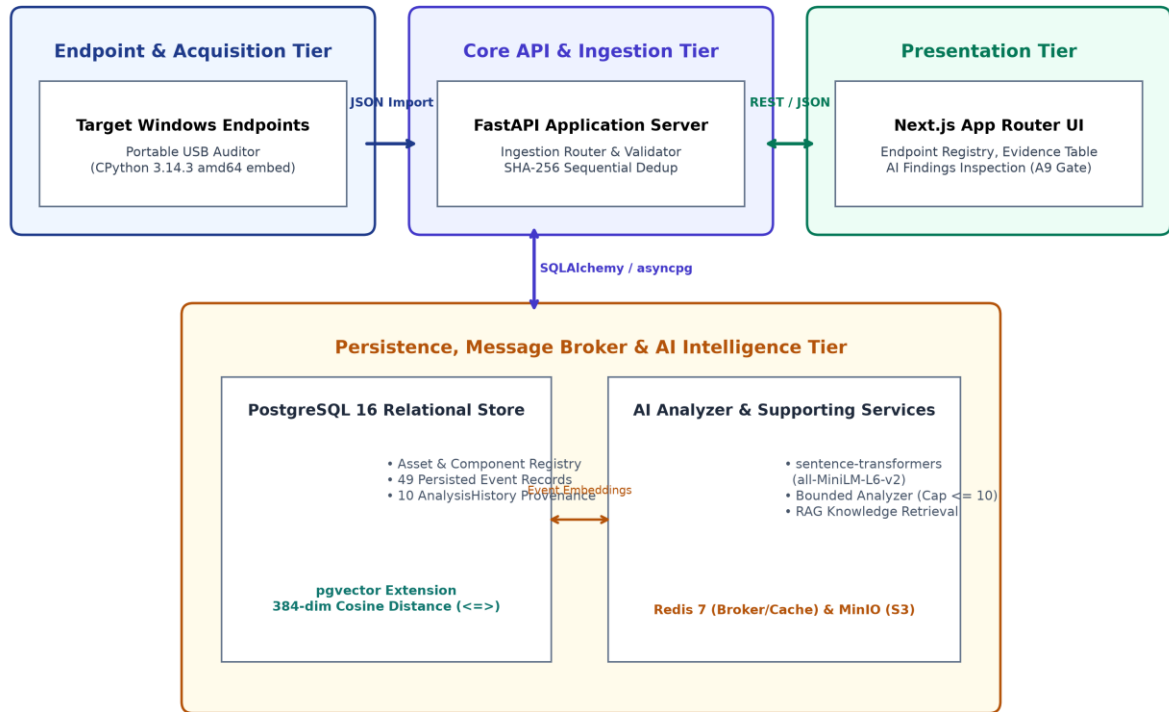
2. Background & Problem Statement

- **Disconnected & Enclave Isolation:** High-security enterprise servers prohibit continuous outbound monitoring connections to central services.
- **Fragile Ephemeral Tracking:** Dynamic DHCP IP churn and host renames disrupt historical telemetry without hardware-anchored identity.
- **Manual Diagnostic Friction:** Ad-hoc scripts produce fragmented diagnostic notes without verifiable provenance across server fleets.
- **Telemetry Volume Overload:** Windows Event Logs generate thousands of routine records daily, obscuring actionable infrastructure faults.

3. Project Objectives (Chapter 1)

- **OBJ-1 Central Asset Registry:** Anchor canonical asset records with cryptographic hardware composite fingerprints across lifecycle changes.
- **OBJ-2 Portable USB Auditor:** Lightweight, non-destructive collection utility executing on embedded CPython 3.14.3.
- **OBJ-3 Offline Ingestion Pipeline:** Schema-validated normalization of offline multipart JSON evidence reports into relational schemas.
- **OBJ-4 Deterministic Deduplication:** Application-level sequential SHA-256 deduplication and severity queue ordering.
- **OBJ-5 Bounded AI Investigation:** pgvector cosine similarity retrieval for prioritized events (bounded workload cap <= 10).
- **OBJ-6 Unified Web Dashboard:** Responsive Next.js 14 App Router UI for asset telemetry, evidence inspection, and AI findings.
- **OBJ-7 Empirical Validation:** End-to-end multi-tier verification from portable USB extraction to live browser display.

4. Multi-Tier System Architecture



- **Hybrid Modular Monolith:** Consolidates collection, ingestion, dedup, and AI triage in a single maintainable service without microservice networking overhead.
- **Persistence & Vector Separation:** PostgreSQL 16 handles ACID relational entities while pgvector computes 384-dimensional cosine distance in unified SQL queries.
- **Decoupled Supporting Tier:** Redis manages message caching/queues, MinIO provides S3-compatible storage, and Next.js 14 delivers hybrid server/client rendering.

5. Technology Implementation Stack

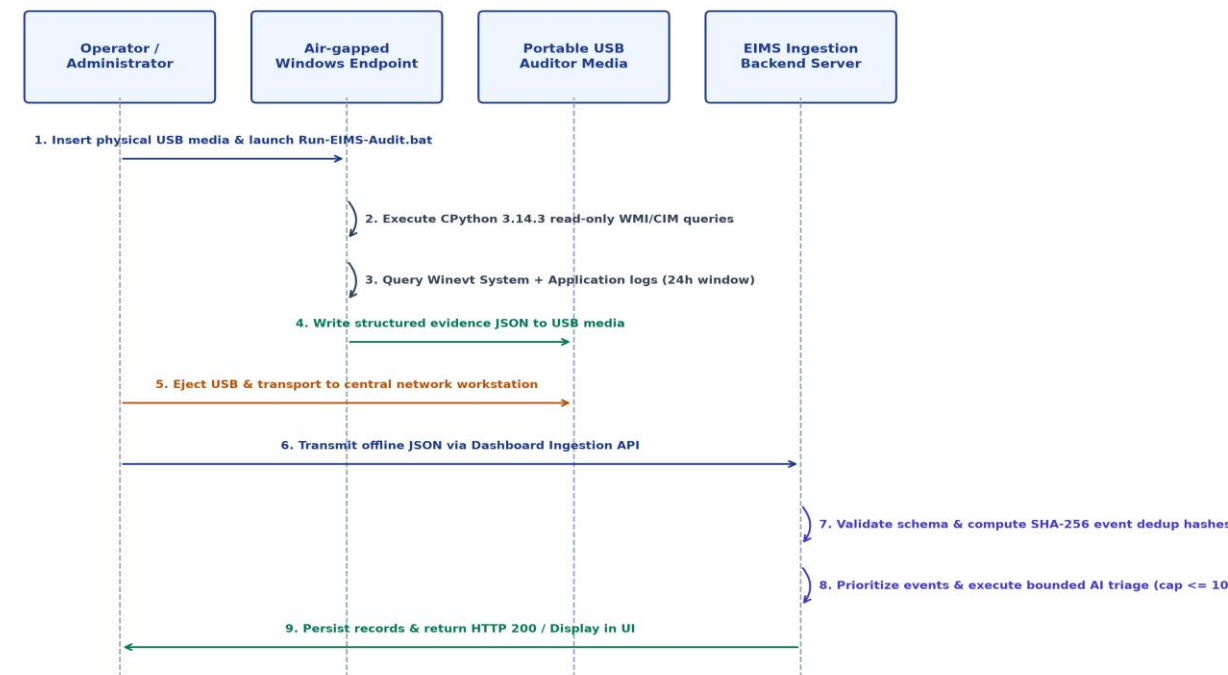
- **Backend API Gateway:** FastAPI + Pydantic + Starlette delivering REST APIs and schema validation.
- **Database & Vector Store:** PostgreSQL 16 with pgvector (384-d dense vector cosine distance search).
- **Broker & Object Store:** Redis 7 (caching/queues) and MinIO (S3-compatible storage for OCR/specs).
- **Operator Web Dashboard:** Next.js 14 App Router, React 18, TypeScript, and Tailwind CSS.
- **Portable USB Auditor:** Embedded CPython 3.14.3 amd64 distribution (no target dependencies).
- **AI Embedding Hierarchy:** FastEmbed BAAI/bge-small-en-v1.5 primary / SentenceTransformer fallback.

6. Event Deduplication & AI Role

DedupKey = SHA-256(asset_id || channel || provider || record_id || occurrence_time)

- **Sequential In-Memory Dedup:** Windows record_id is monotonically assigned; SHA-256 hash enables deterministic duplicate skipping across sequential imports.
- **Concurrency Boundary:** Enforced sequentially in memory. Concurrent identical imports may race as database-level uniqueness is not currently enforced.
- **Severity Priority Ranking:** Candidate events prioritized: Critical (Level 1) > Error (Level 2) > Warning (Level 3) > Info (Level 4).
- **Bounded Workload Cap:** Analyzer strictly triages at most 10 selected new events per import, protecting worker threads from load spikes.
- **Assistive Role Boundary:** AI provides diagnostic hypotheses and suggested verification checks; does NOT execute autonomous remediation or guaranteed RCA.

7. Portable Offline Workflow



- **Bundled Portable Runtime:** Embedded CPython 3.14.3 amd64; requires NO preinstalled Python, pip, Git, or Docker on target hosts.
- **Non-Destructive Posture:** Read-only WMI/CIM and event queries; writes reports/logs strictly to configured USB directories.
- **Scoped Telemetry Window:** Captures System and Application logs over 24h (default maximum 500 events across combined query).
- **Deterministic JSON Output:** Produces structured, schema-compliant JSON payloads for central offline import.

8. Validation Results & Metrics

37 / 0 / 4 Hermetic Test Suite	49 Events Persisted Telemetry
10 Records AnalysisHistory Records	HTTP 200 Filtered History API
Gate A9 PASS Browser Runtime Gate	0 Errors Frontend Quality Gates

- **Workload Bounding Distinction:** The 49 events represent persisted evidence in the database. 10 AnalysisHistory records were created associated with the selected field dataset, matching the analyzer's strict cap of at most 10 selected new events per import.
- **Validation Scope Boundary:** Earlier field trials verified physical USB execution and JSON extraction. The latest on-site session verified downstream data integrity (persisted DB rows, filtered API, and UI) without re-running physical USB collection.
- **A9 Runtime Verification:** The field dataset contained 49 persisted Windows Event Evidence rows. The A9 browser check confirmed that Event Evidence and AI Findings were both available in the endpoint investigation view.

9. Limitations & Future Work

Scoped Limitations:

- Windows-first collection focus (WMI/winevt); System and Application channels only.
- Sequential deduplication in memory; concurrent identical imports may race.
- Prototype baseline; does not include multi-region HA or enterprise SSO.

Future Engineering Work:

- Database-level concurrency-safe deduplication (PostgreSQL composite UNIQUE constraints).
- Cross-endpoint event correlation (multi-host temporal incident clustering).
- Production reliability and identity integration (clustering, SAML 2.0 / OIDC SSO).

10. Conclusion & Approval

Conclusion:

- EIMS centralizes asset inventory and operational evidence through hardware fingerprints.
- Portable collection safely acquires air-gapped evidence via embedded CPython 3.14.3.
- Combined validation evidence covered portable acquisition/import behavior and the downstream database, API, analysis-history, and UI path.

Acknowledgment:

Sincere gratitude to KANEMATSU ELECTRONICS (THAILAND) for host enterprise support, Workplace Supervisor Mr. Chalermpol Phuangchalam for professional guidance, and Academic Supervisor Asst. Prof. Dr. Phongsak Keeratiwintakorn for academic mentorship.

Asst. Prof. Dr. Phongsak Keeratiwintakorn (Project Advisor)
Date: ____/____/2026